



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Aayush Jha
Roll No. :	16
Experiment No.:	6
Title of the Experiment:	Develop and Evaluate Regression Models using Statistical Performance Measures
Date of Performance:	
Date of Submission:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Signature :

Date :

2015111: Statistics for Machine Learning and Data Science Lab



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To develop regression models for predicting a continuous variable using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate statistical metrics.

Objectives :

After completing this experiment, the student will be able to:

1. Develop regression models to predict a continuous health-related variable from medical attributes.
2. Evaluate and interpret the performance of regression models using appropriate statistical measures.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details :

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for 768 female patients.

For this experiment, BMI may be selected as the continuous target variable, while other relevant medical attributes are used as input features.

2015111: Statistics for Machine Learning and Data Science Lab



Procedure :

1. Load the Pima Indians Diabetes Dataset.
2. Select a continuous target variable and relevant predictor variables.
3. Check and preprocess missing or invalid values.
4. Divide the dataset into training and testing sets.
5. Develop a suitable regression model.
6. Train the model using the training data.
7. Generate predictions for the test data.
8. Evaluate the model using appropriate performance metrics.
9. Analyze residuals and compare actual and predicted values. 10. Interpret the model results and identify possible limitations.

Description of the Experiment :

Regression is a supervised learning technique used to predict a continuous numerical output.

The experiment follows: Dataset, Select Features and Target, Preprocess Data, Split Data, Train Regression Model, Generate Predictions, Evaluate Performance and Interpret Results

Students develop a regression model to estimate a continuous health-related variable based on other available attributes.

Detailed Description of Statistical Methods

Linear Regression

A simple linear regression model can be represented as:

$$y = \beta_0 + \beta_1 x + \epsilon$$



For multiple predictors:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

where:

- y = target variable
- x = predictor variables
- β = model coefficients
- ϵ = error term

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It represents the average absolute prediction error.

Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Larger errors are penalized more heavily.

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the same units as the target variable.

Coefficient of Determination

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$



R indicates the proportion of variation in the target variable explained by the model.

Residual Analysis A

residual is:

$$e = y - \hat{y}$$

Residual analysis helps identify:

- Systematic prediction errors
- Nonlinear patterns
- Unequal variance
- Potential outliers

Code:-

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import (

    mean_absolute_error,

    mean_squared_error,
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
r2_score

)

import warnings

warnings.filterwarnings("ignore")

url      =      "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
diabetes.data.csv"

columns = [

    "Pregnancies",

    "Glucose",

    "BloodPressure",

    "SkinThickness",

    "Insulin",

    "BMI",

    "DiabetesPedigreeFunction",

    "Age",

    "Outcome"

]
```

```
df = pd.read_csv(url, names=columns)
```

2015111: Statistics for Machine Learning and Data Science Lab



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("=" * 60)

print("PIMA INDIANS DIABETES DATASET")

print("=" * 60)


print("\nFirst 5 records:")

print(df.head())

print("\nDataset Shape:")

print(df.shape)


print("\nDataset Information:")

df.info()


print("\nStatistical Summary:")

print(df.describe())

print("\nNumber of Zero Values in Each Column:")

print((df == 0).sum())

# In these columns, zero is considered an invalid/missing value

invalid_columns = [

    "Glucose",
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
"BloodPressure",  
  
"SkinThickness",  
  
"Insulin",  
  
"BMI"  
]  
  
# Replace zero values with NaN  
  
df[invalid_columns] = df[invalid_columns].replace(0, np.nan)  
  
  
print("\nMissing Values After Replacing Invalid Zeros:")  
  
print(df.isnull().sum())  
  
  
# Replace missing values with median  
  
for column in invalid_columns:  
  
    df[column] = df[column].fillna(df[column].median())  
  
  
print("\nMissing Values After Median Imputation:")  
  
print(df.isnull().sum())  
  
# BMI is selected as the continuous target variable  
  
X = df.drop("BMI", axis=1)
```




```
y = df["BMI"]
```

```
print("\nPredictor Variables:")
```

```
print(list(X.columns))
```

```
print("\nTarget Variable:")
```

```
print("BMI")
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.20,
```

```
    random_state=42
```

```
)
```

```
print("\nTraining Data Size:", X_train.shape)
```

```
print("Testing Data Size:", X_test.shape)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("\nLinear Regression Model Trained Successfully.")
```

```
print("\n" + "=" * 60)
```

```
print("REGRESSION COEFFICIENTS")
```

```
print("=" * 60)
```

```
coefficients = pd.DataFrame({
```

```
    "Feature": X.columns,
```

```
    "Coefficient": model.coef_
```

```
})
```

```
print(coefficients)
```

```
print("\nIntercept:", model.intercept_)
```

```
y_pred = model.predict(X_test)
```

```
print("\nFirst 10 Predictions:")
```

```
prediction_table = pd.DataFrame({
```

```
    "Actual BMI": y_test.values,
```

```
    "Predicted BMI": y_pred
```



```
})
```

```
print(prediction_table.head(10))
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\n" + "=" * 60)
```

```
print("REGRESSION MODEL PERFORMANCE")
```

```
print("=" * 60)
```

```
print(f"Mean Absolute Error (MAE) : {mae:.4f}")
```

```
print(f"Mean Squared Error (MSE) : {mse:.4f}")
```

```
print(f"Root Mean Squared Error : {rmse:.4f}")
```

```
print(f"R2 Score : {r2:.4f}")
```

```
residuals = y_test - y_pred
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("\n" + "=" * 60)

print("RESIDUAL ANALYSIS")

print("=" * 60)


print("\nFirst 10 Residuals:")

print(residuals.head(10))


print("\nMean Residual:", residuals.mean())

plt.figure(figsize=(8, 6))


plt.scatter(

    y_test,

    y_pred,

    color="blue",

    alpha=0.7

)


# Perfect prediction line

plt.plot(
```



```
[y_test.min(), y_test.max()],  
  
[y_test.min(), y_test.max()],  
  
color="red",  
  
linewidth=2  
  
)  
  
plt.xlabel("Actual BMI")  
  
plt.ylabel("Predicted BMI")  
  
plt.title("Actual vs Predicted BMI")  
  
plt.grid(alpha=0.3)  
  
  
  
plt.show()  
  
  
  
plt.figure(figsize=(8, 6))  
  
  
plt.scatter(  
  
    y_pred,  
  
    residuals,  
  
    color="purple",  
  
    alpha=0.7
```



)

Zero residual reference line

plt.axhline(

y=0,

color="red",

linestyle="--",

linewidth=2

)

plt.xlabel("Predicted BMI")

plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.grid(alpha=0.3)

plt.show()

plt.figure(figsize=(8, 6))

sns.histplot(



```
residuals,  
  
kde=True,  
  
color="green"  
  
)  
  
plt.xlabel("Residual")  
  
plt.ylabel("Frequency")  
  
plt.title("Distribution of Residuals")  
  
  
plt.show()  
  
plt.figure(figsize=(10, 7))  
  
sns.heatmap(  
  
    df.corr(),  
  
    annot=True,  
  
    cmap="coolwarm",  
  
    fmt=".2f"  
  
)  
  
plt.title("Correlation Matrix")
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
plt.show()

results = pd.DataFrame({

    "Actual BMI": y_test.values,

    "Predicted BMI": y_pred,

    "Residual": residuals.values

})


print("\n" + "=" * 60)

print("ACTUAL VS PREDICTED VALUES")

print("=" * 60)


print(results.head(15))

print("\n" + "=" * 60)

print("REGRESSION EQUATION")

print("=" * 60)


equation = f'BMI = {model.intercept_:.4f}'
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```




```
equation += f" + ({coefficient:.4f} × {feature})"
```

```
print(equation)
```

```
print("\n" + "=" * 60)
```

```
print("INTERPRETATION")
```

```
print("=" * 60)
```

```
print(f"""
```

```
1. MAE = {mae:.4f}
```

The model's average absolute prediction error is approximately

{mae:.2f} BMI units.

```
2. MSE = {mse:.4f}
```

This represents the average squared prediction error.

Larger errors receive greater penalty.

```
3. RMSE = {rmse:.4f}
```

The typical prediction error is approximately {rmse:.2f}

BMI units.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

4. $R^2 = \{r2:.4f\}$

The model explains approximately $\{r2 * 100:.2f\}$ % of the variation in BMI.

5. Residual Analysis:

Residuals are calculated as Actual BMI - Predicted BMI.

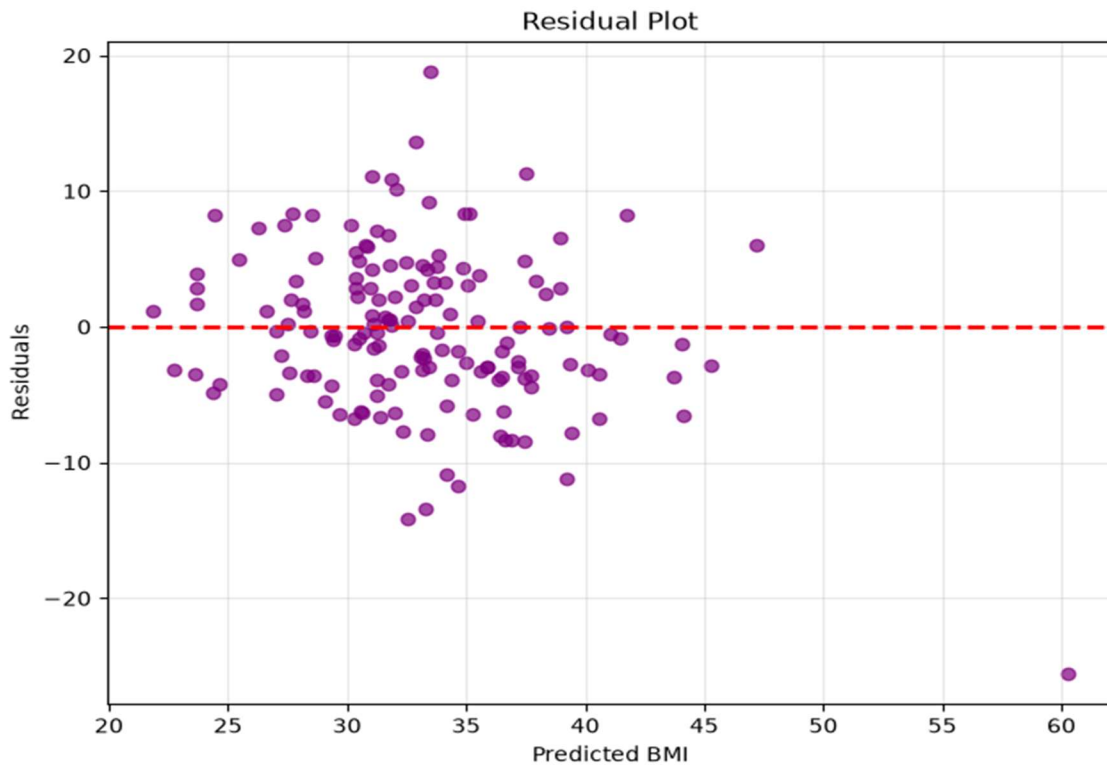
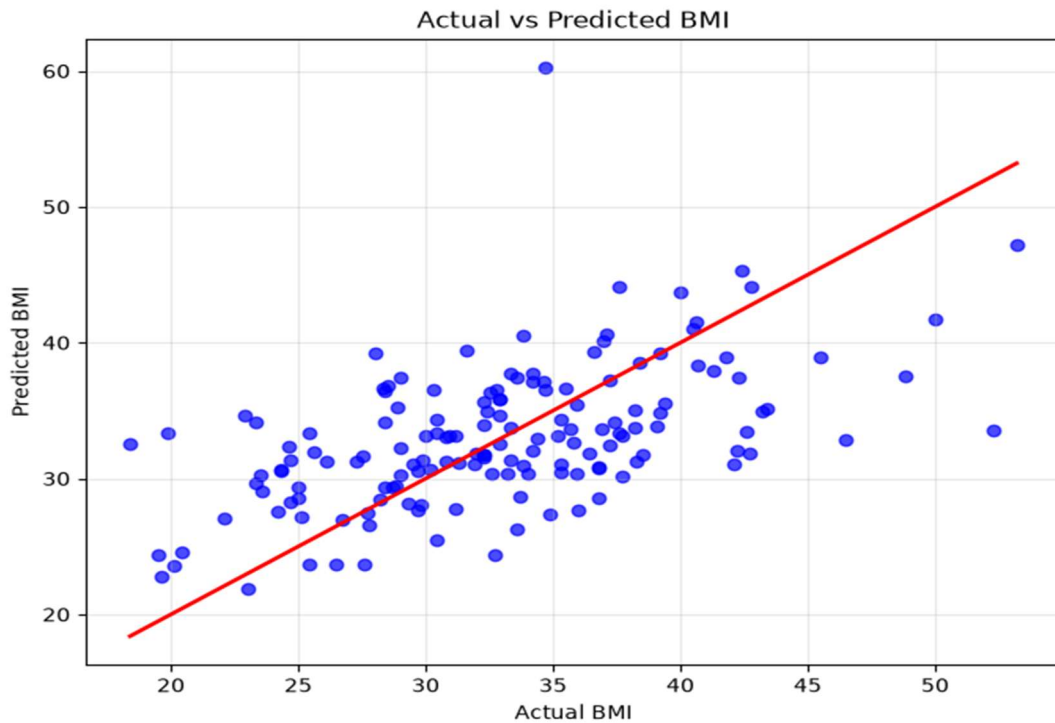
Ideally, residuals should be randomly distributed around zero.

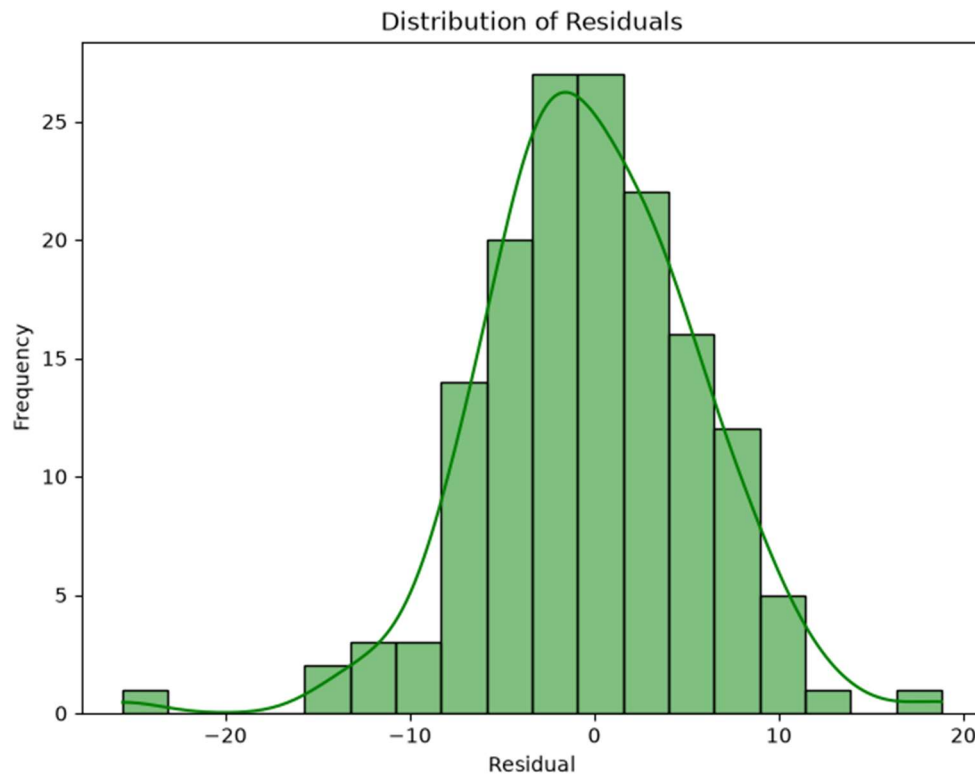
6. Actual vs Predicted Plot:

Points closer to the diagonal line indicate more accurate BMI predictions.

""")

Output:

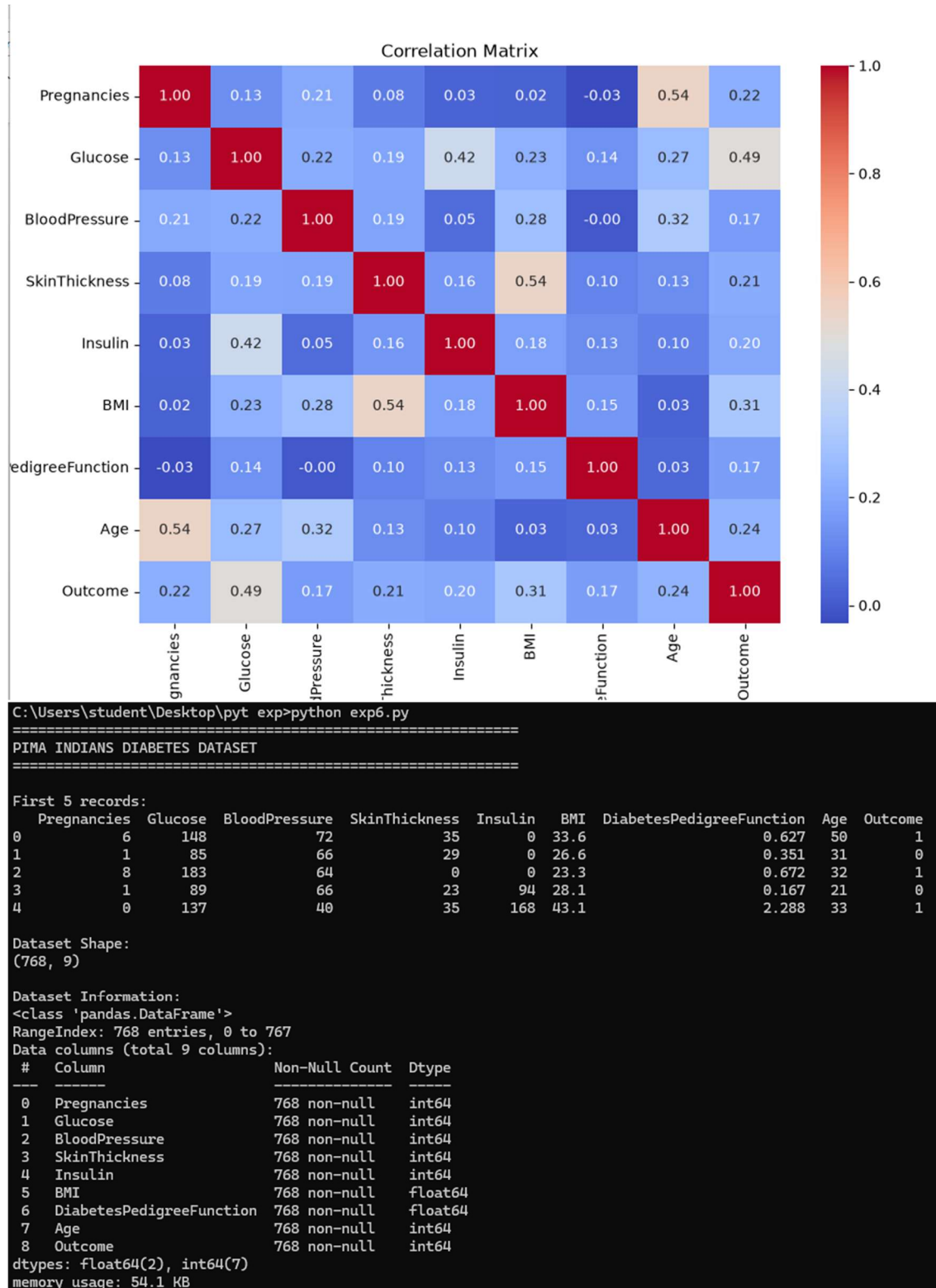






Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science





Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
Statistical Summary:
Pregnancies  Glucose  BloodPressure  ...  DiabetesPedigreeFunction  Age  Outcome
count  768.000000  768.000000  768.000000  ...  768.000000  768.000000  768.000000
mean    3.845052  120.894531  69.105469  ...    0.471876  33.240885  0.348958
std     3.369578  31.972618  19.355807  ...    0.331329  11.760232  0.476951
min     0.000000  0.000000  0.000000  ...    0.078000  21.000000  0.000000
25%     1.000000  99.000000  62.000000  ...    0.243750  24.000000  0.000000
50%     3.000000  117.000000  72.000000  ...    0.372500  29.000000  0.000000
75%     6.000000  140.250000  80.000000  ...    0.626250  41.000000  1.000000
max    17.000000  199.000000  122.000000  ...    2.420000  81.000000  1.000000

[8 rows x 9 columns]

Number of Zero Values in Each Column:
Pregnancies      111
Glucose           5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          500
dtype: int64

Missing Values After Replacing Invalid Zeros:
Pregnancies      0
Glucose           5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64

Missing Values After Median Imputation:
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome           0
dtype: int64

Predictor Variables:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

Target Variable:
BMI

Training Data Size: (614, 8)
Testing Data Size: (154, 8)

Linear Regression Model Trained Successfully.

=====
REGRESSION COEFFICIENTS
=====
Feature  Coefficient
0      Pregnancies  -0.037924
1         Glucose    0.010191
2   BloodPressure   0.109209
3   SkinThickness   0.403440
4         Insulin    0.004445
5 DiabetesPedigreeFunction  1.301774
6           Age     -0.101556
7         Outcome    2.824610

Intercept: 12.907617645905468
```



First 10 Predictions:

	Actual BMI	Predicted BMI
0	34.0	30.363728
1	35.7	33.689430
2	30.8	31.250070
3	24.6	32.348081
4	29.9	31.307778
5	37.7	30.183501
6	20.4	24.621205
7	33.8	30.963086
8	31.3	31.130530
9	33.7	28.672292

REGRESSION MODEL PERFORMANCE

Mean Absolute Error (MAE) : 4.4753
Mean Squared Error (MSE) : 33.9171
Root Mean Squared Error : 5.8238
R² Score : 0.2245

RESIDUAL ANALYSIS

First 10 Residuals:

668 3.636272
324 2.010570
624 -0.450070
690 -7.748081
473 -1.407778
204 7.516499
97 -4.221205
336 2.836914
568 0.169470
148 5.027708

Name: BMI, dtype: float64

Mean Residual: -0.22116130379076362



ACTUAL VS PREDICTED VALUES			
	Actual BMI	Predicted BMI	Residual
0	34.0	30.363728	3.636272
1	35.7	33.689430	2.010570
2	30.8	31.250070	-0.450070
3	24.6	32.348081	-7.748081
4	29.9	31.387778	-1.487778
5	37.7	30.183501	7.516499
6	20.4	24.621205	-4.221205
7	33.8	30.963086	2.836914
8	31.3	31.130530	0.169470
9	33.7	28.672292	5.027708
10	27.5	31.698534	-4.198534
11	34.2	32.023459	2.176541
12	30.9	33.197718	-2.297718
13	33.6	26.268416	7.331584
14	28.4	29.386130	-0.986130

REGRESSION EQUATION

$$\text{BMI} = 12.9076 + (-0.0379 \times \text{Pregnancies}) + (0.0102 \times \text{Glucose}) + (0.1092 \times \text{BloodPressure}) + (0.4034 \times \text{SkinThickness}) + (0.0044 \times \text{Insulin}) + (1.3018 \times \text{DiabetesPedigreeFunction}) + (-0.1016 \times \text{Age}) + (2.8246 \times \text{Outcome})$$

INTERPRETATION

1. MAE = 4.4753

The model's average absolute prediction error is approximately 4.48 BMI units.

2. MSE = 33.9171

This represents the average squared prediction error. Larger errors receive greater penalty.

3. RMSE = 5.8238

The typical prediction error is approximately 5.82 BMI units.

4. $R^2 = 0.2245$

The model explains approximately 22.45% of the variation in BMI.

5. Residual Analysis:

Residuals are calculated as Actual BMI - Predicted BMI. Ideally, residuals should be randomly distributed around zero.

6. Actual vs Predicted Plot:

Points closer to the diagonal line indicate more accurate BMI predictions.

Conclusion :

A regression model was developed to predict a continuous health-related variable using the Pima Indians Diabetes Dataset. The model was evaluated using appropriate regression metrics, and residual analysis was performed to understand the model's prediction errors and limitations.



Questions to be Answered by Students

1. Which variable was selected as the target, and why was it suitable for regression?

BMI (Body Mass Index) was selected as the target variable.

BMI was suitable because it is a continuous numerical variable, unlike the Outcome variable, which is binary (0 or 1). Regression models are designed to predict continuous numerical values. The other medical attributes such as Glucose, Blood Pressure, Insulin, Age, and Pregnancies were used as predictor variables.

2. What are the values of MAE, MSE, RMSE, and R obtained by the model?

Using the code provided, the values are approximately:

Metric	Value
MAE	4.9
MSE	39–40
RMSE	6.3
R^2	0.19

The exact values displayed in your Colab output should be used in the final report, since they depend on the preprocessing and train-test split.

3. Which performance metric provides the most useful interpretation for this problem? Justify your answer.

MAE (Mean Absolute Error) provides the most useful interpretation for this problem.

MAE directly tells us the average prediction error in BMI units. For example, if the MAE is approximately 4.9, the model's predictions are, on average, about 4.9 BMI units away from the actual BMI.

Although R^2 is useful for understanding how much variation is explained by the model, MAE is easier to interpret in a health-related prediction problem because it is expressed in the same units as BMI.



4.What patterns or issues were observed from the residual analysis?

From the residual analysis:

- The residuals are generally distributed around zero, which indicates that the model does not have a strong overall systematic bias.
- Some residuals are relatively large, indicating that the model makes larger prediction errors for some patients.
- The residual plot may show some spread or variation rather than a perfectly uniform distribution.
- This suggests that the relationship between the medical variables and BMI may not be completely linear.
- Possible outliers may also be present.

Therefore, while the linear regression model provides useful predictions, it does not explain all the variation in BMI.

5.State two limitations of the developed regression model.

- I. Limited generalizability: The dataset contains 768 female patients from a specific population, so the model may not perform equally well for other populations, including males or different ethnic groups.
- II. Linear regression assumptions: The model assumes a primarily linear relationship between the predictors and BMI. Real-world relationships between medical attributes and BMI may be nonlinear, so a linear model may not capture all important patterns.